

Security Assessment Report - Frontend

Project: AWS Pipeline Demo - PLOCLO CMS Frontend
Scan Date: 2025
Scan Scope: d:\dev\aws-pipeline-demo\PLOCLO_cms\frontend
Tool: Amazon Q Developer Code Review

Executive Summary

การสแกนความปลอดภัยพบช่องโหว่ 22 รายการ แบ่งเป็น:

- Critical: 3 รายการ
- High: 9 รายการ
- Medium: 10 รายการ

ความเสี่ยงหลัก

- Remote Code Execution (RCE) ใน React Server Components
- Prototype Pollution ใน flattened package
- Cross-Site Scripting (XSS) ใน 2 ไฟล์
- Log Injection ใน 4 จุด
- Sensitive Data Exposure (JWT ใน localStorage)
- Vulnerable Dependencies 16 packages

● Critical Vulnerabilities (3)

1. React Server Components - Pre-Auth RCE

- File: package-lock.json (line 18-19)
- CWE: CWE-502 (Deserialization of Untrusted Data)
- Severity: Critical
- Impact: Remote Code Execution ก่อน authentication
- Affected Versions: React 19.0.0, 19.1.0, 19.1.1, 19.2.0
- Packages: react-server-dom-parcel, react-server-dom-turbopack, react-server-dom-webpack

Description:

ช่องโหว่ใน React Server Components ที่ deserialize payload จาก HTTP request โดยไม่ปลอดภัย ทำให้ attacker สามารถ execute arbitrary code ได้

Remediation:

```
npm update react react-dom
```

2. Flatted - Prototype Pollution

- File: package-lock.json (line 3861-3862)
- CWE: CWE-915, CWE-1321 (Prototype Pollution)
- Severity: Critical
- Impact: Global prototype pollution → RCE potential

Description:

flatted.parse() ใช้ key __proto__ จาก JSON โดยตรงโดยไม่ validate ทำให้สามารถ pollute Array.prototype ได้

Remediation:

```
npm install flatted@^3.4.2
```

3. Preact - HTML Injection → XSS

- File: package-lock.json (line 5543-5544)
- CWE: CWE-843 (Type Confusion)
- Severity: Critical
- Impact: HTML injection → XSS execution
- Affected Versions: Preact 10.26.5+

Description:

Regression ใน Preact ทำให้ JSON payload ที่ crafted พิเศษสามารถถูก treat เป็น VNode ได้ นำไปสู่ HTML injection

Remediation:

```
npm update preact
```

● High Vulnerabilities (9)

Code-Level Issues

4. Cross-Site Scripting (XSS) - PerformanceTable.tsx

- File: app/viewChart/viewChartComponent/PerformanceTable.tsx (line 31-32)
- CWE: CWE-79, CWE-80

- **Severity:** High

Vulnerable Code:

```
// User input rendered without sanitization
<div dangerouslySetInnerHTML={{ __html: userData }} />
```

Remediation:

```
import DOMPurify from 'dompurify';

// Sanitize before rendering
<div dangerouslySetInnerHTML={{
  __html: DOMPurify.sanitize(userData)
}} />
```

5. Sensitive Data in localStorage - api.ts

- **File:** utils/api.ts (line 28-29)
- **CWE:** CWE-79, CWE-80
- **Severity:** High
- **Impact:** JWT/tokens accessible via XSS

Vulnerable Code:

```
localStorage.setItem('token', jwtToken);
```

Remediation:

```
// ❌ httpOnly cookie แทน localStorage
// Backend ต้อง set cookie:
// Set-Cookie: token=xxx; HttpOnly; Secure; SameSite=Strict

// Frontend ไม่ต้องเก็บ token เอง
// Browser จะเก็บ cookie อัตโนมัติ
```

6-9. Log Injection (4 จุด)

- **CWE:** CWE-117
- **Severity:** High
- **Impact:** Log forging, log monitoring bypass

Affected Files:

1. app/editCourse/assignmentMapping.tsx (line 179-180)
2. app/editCourse/clopioMapping.tsx (line 66-67)
3. app/editCourse/clopioMapping.tsx (line 105-106)
4. app/editProgram/page.tsx (line 101-102)

Vulnerable Pattern:

```
console.log("User input: " + userInput);
```

Remediation:

```
// Sanitize newlines ก่อน log
const sanitize = (input: string) => input.replace(/\r\n/g, '');
console.log("User input: " + sanitize(userInput));
```

Package Vulnerabilities

10. SheetJS - ReDoS

- **File:** package-lock.json (line 32-33)
- **CWE:** CWE-1333 (Regular Expression DoS)
- **Severity:** High
- **Fix:** npm install xlsx@^0.20.2

11. node-tar - Path Traversal

- **File:** package-lock.json (line 1213-1214)
- **CWE:** CWE-22 (Path Traversal)
- **Severity:** High
- **Impact:** Arbitrary file read/write
- **Fix:** npm install tar@^7.5.8

12. ajv - ReDoS

- **File:** package-lock.json (line 205-206)
- **CWE:** CWE-400, CWE-1333
- **Severity:** High
- **Fix:** npm install ajv@^8.18.0

13. i18next-fs-backend - Path Traversal

- **File:** package-lock.json (line 4308-4309)
- **CWE:** CWE-22, CWE-73
- **Severity:** High
- **Impact:** Arbitrary file read via ?lng= parameter
- **Fix:** npm install i18next-fs-backend@^2.6.4

14. brace-expansion - Infinite Loop DoS

- **File:** package-lock.json (line 1793-1794)
- **CWE:** CWE-400
- **Severity:** High
- **Fix:** npm install brace-expansion@^5.0.5

15. minimatch - ReDoS

- **File:** package-lock.json (line 169-170)
- **CWE:** CWE-1333
- **Severity:** High
- **Fix:** npm install minimatch@^10.2.3

🔴 Medium Vulnerabilities (10)

16. i18next-http-backend - URL Injection

- **File:** package-lock.json (line 20-21)
- **CWE:** CWE-22, CWE-74
- **Severity:** Medium
- **Fix:** npm install i18next-http-backend@^3.0.5

17. postcss - XSS via </style> Escape

- **File:** package-lock.json (line 37-38)
- **CWE:** CWE-79
- **Severity:** Medium
- **Fix:** npm install postcss@^8.5.10

18. axios - Body Length Bypass

- **File:** package-lock.json (line 13-14)
- **CWE:** CWE-770
- **Severity:** Medium
- **Fix:** npm install axios@^1.15.1

19. next-intl - Prototype Pollution

- **File:** package-lock.json (line 25-26)
- **CWE:** CWE-1321
- **Severity:** Medium
- **Fix:** npm install next-intl@latest

20. follow-redirects - Header Leakage

- **File:** package-lock.json (line 2457-2458)
- **CWE:** CWE-200 (Information Exposure)
- **Severity:** Medium
- **Impact:** Custom auth headers (X-API-Key) leaked on redirect
- **Fix:** npm install follow-redirects@latest

21. picomatch - Method Injection

- **File:** package-lock.json (line 5318-5319)
- **CWE:** CWE-624, CWE-1321
- **Severity:** Medium
- **Fix:** npm install picomatch@^4.0.4

Remediation Plan

Phase 1: Immediate (Critical)

```
# อัปเดต dependencies ที่มีช่องโหว่ Critical
npm install react@latest react-dom@latest
npm install flatted@^3.4.2
npm install preact@latest
```

Phase 2: High Priority (1-2 วัน)

```
# อัปเดต packages ที่มีช่องโหว่ High
npm install xlsx@^0.20.2 tar@^7.5.8 ajv@^8.18.0
npm install i18next-fs-backend@^2.6.4
npm install brace-expansion@^5.0.5 minimatch@^10.2.3
```

บทนำ Code:

- เพิ่ม DOMPurify ใน PerformanceTable.tsx
- ย้าย JWT จาก localStorage → httpOnly cookie
- Sanitize input ใน 4 ไฟล์ที่ Log Injection

Phase 3: Medium Priority (1 สัปดาห์)

```
# อัปเดต packages ที่เหลือ
npm audit fix
npm update
```

Phase 4: Verification

```
# ตรวจสอบหลังแก้ไข
npm audit
npm outdated
```

Security Best Practices

1. Input Validation

```
// ใช้ validation library
import { z } from 'zod';

const userSchema = z.object({
  name: z.string().max(100),
  email: z.string().email()
});
```

2. Output Encoding

```
// ใช้ DOMPurify ล้าง HTML
import DOMPurify from 'dompurify';
const clean = DOMPurify.sanitize(dirty);
```

3. Secure Token Storage

```
// Backend: Set httpOnly cookie
res.cookie('token', jwt, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 3600000
});

// Frontend: ไม่ต้องจัดการ token
// Browser จะส่ง cookie อัตโนมัติ
```

4. Content Security Policy

```
// next.config.ts
const securityHeaders = [
  {
    key: 'Content-Security-Policy',
    value: "default-src 'self'; script-src 'self'"
  }
];
```

5. Dependency Management

```
# ตรวจสอบ vulnerabilities เป็นประจำ
npm audit
npm outdated

# ใช้ Dependabot หรือ Renovate
# เพื่ออัปเดต dependencies อัตโนมัติ
```

Testing Recommendations

1. Security Testing

- ใช้ OWASP ZAP หรือ Burp Suite ทดสอบ XSS
- ทดสอบ Log Injection ด้วย payload: \r\nFAKE_LOG_ENTRY
- ทดสอบ Path Traversal: ?lng=../../etc/passwd

2. Automated Scanning

```
# เพิ่มใน CI/CD pipeline
npm audit --audit-level=high
```

3. Code Review Checklist

- [] ไม่มี dangerouslySetInnerHTML โดยไม่ sanitize
- [] ไม่เก็บ sensitive data ใน localStorage
- [] Sanitize input ก่อน log

- [] Validate user input ၇၇၃၈
- [] ၇၇ parameterized queries (၈၈၈ SQL)

References

- [OWASP Top 10 \(https://owasp.org/www-project-top-ten/\)](https://owasp.org/www-project-top-ten/)
- [CWE Top 25 \(https://cwe.mitre.org/top25/\)](https://cwe.mitre.org/top25/)
- [React Security Best Practices \(https://react.dev/learn/security\)](https://react.dev/learn/security)
- [AWS Security Best Practices \(https://aws.amazon.com/security/best-practices/\)](https://aws.amazon.com/security/best-practices/)

Appendix: Full Vulnerability List

#	File	Line	Severity	CWE	Issue
1	package-lock.json	18-19	Critical	CWE-502	React RCE
2	package-lock.json	3861-3862	Critical	CWE-915	Flatted Prototype Pollution
3	package-lock.json	5543-5544	Critical	CWE-843	Preact HTML Injection
4	PerformanceTable.tsx	31-32	High	CWE-79	XSS
5	api.ts	28-29	High	CWE-79	Sensitive Data Exposure
6	assignmentMapping.tsx	179-180	High	CWE-117	Log Injection
7	cloploMapping.tsx	66-67	High	CWE-117	Log Injection
8	cloploMapping.tsx	105-106	High	CWE-117	Log Injection
9	editProgram/page.tsx	101-102	High	CWE-117	Log Injection
10	package-lock.json	32-33	High	CWE-1333	SheetJS ReDoS
11	package-lock.json	1213-1214	High	CWE-22	node-tar Path Traversal
12	package-lock.json	205-206	High	CWE-1333	ajv ReDoS
13	package-lock.json	4308-4309	High	CWE-22	i18next-fs Path Traversal
14	package-lock.json	1793-1794	High	CWE-400	brace-expansion DoS
15	package-lock.json	169-170	High	CWE-1333	minimatch ReDoS
16	package-lock.json	20-21	Medium	CWE-22	i18next-http URL Injection
17	package-lock.json	37-38	Medium	CWE-79	postcss XSS
18	package-lock.json	13-14	Medium	CWE-770	axios Body Length Bypass
19	package-lock.json	25-26	Medium	CWE-1321	next-intl Prototype Pollution
20	package-lock.json	2457-2458	Medium	CWE-200	follow-redirects Header Leak
21	package-lock.json	5318-5319	Medium	CWE-1321	picomatch Method Injection

Report Generated by: Amazon Q Developer
Contact: DevSecOps Team